SANI VARADA

AI/ML Engineer · Quantitative Analyst · LLM & Automation Architect · Founder, ML Arc

Hyderabad, Telangana | founder@mlarcai.com | linkedin.com/in/sani-varada-845869176

+91-7021628334

PROFESSIONAL SUMMARY

AI/ML Engineer, Quantitative Analyst, and Entrepreneur combining deep technical expertise with financial markets acumen. Founder of ML Arc — an AI software agency building LLM-powered SaaS platforms, RAG pipelines, conversational AI agents, and intelligent automation for global clients. Applies quantitative modeling, algorithmic strategy, and data-driven analysis across financial instruments including XAU/USD, forex majors, BTC/USD, BankNifty, and Nifty50. Proficient in Python, LangChain, Azure, Databricks, and end-to-end MLOps observability with Grafana. Bridges the gap between cutting-edge AI research and real-world revenue impact.

CORE COMPETENCIES & TECHNICAL SKILLS

AI / LLM: LangChain · LlamaIndex · OpenAI API · RAG Pipelines · Prompt Engineering · Vector DBs (Pinecone, FAISS, ChromaDB) · Fine-tuning · LangGraph · LangSmith

Quantitative & Finance: Algorithmic Trading · Quantitative Strategy Development · Statistical Arbitrage · Time Series Analysis · Technical Analysis · Risk Management · Portfolio Optimization · Backtesting (Backtrader, Zipline) · Options Greeks · Monte Carlo Simulation

Languages & Data: Python (Pandas, NumPy, Scikit-learn, TA-Lib, PyTorch) · SQL · R · JavaScript/TypeScript · Bash

Cloud & Data Engineering: Microsoft Azure (ADF, Functions, Blob, AKS, Azure ML) · Databricks · Apache Spark · Delta Lake · MLflow · dbt · Airflow

MLOps & Observability: Grafana · Prometheus · OpenTelemetry · Docker · Docker Compose · Docker Swarm · Kubernetes (AKS) · Container Registry · CI/CD (GitHub Actions, Azure DevOps) · MLflow · Weights & Biases

Financial Instruments: XAU/USD · XAG/USD · BTC/USD · EUR/USD · GBP/USD · BankNifty · Nifty50 · Forex Majors · Commodities · Crypto

Web & SaaS: React · Next.js · Node.js · FastAPI · REST APIs · Supabase · Firebase · Vercel · Streamlit

AI Agents & Automation: Multi-agent Orchestration · Voice Agents (ElevenLabs, Twilio) · Chatbot Development · n8n · Make · Zapier

Marketing & Growth: Meta Ads Manager · Google Ads API · Performance Marketing · ROAS Optimization · CRO

PROFESSIONAL EXPERIENCE

Founder & AI Solutions Architect · ML Arc *2026 – Present | Hyderabad, India*

- Founded ML Arc, an AI software agency delivering LLM-powered SaaS, automation agents, voice chatbots, and performance marketing solutions to global clients.
- Architected production RAG pipelines, multi-agent LangChain systems, and custom fine-tuned models reducing client operational overhead by 60% on average.
- Led delivery of 15+ AI projects end-to-end — from requirements through deployment and MLOps monitoring — maintaining 100% on-time delivery record.
- Managed Meta and Google Ads campaigns generating 4x+ average ROAS for e-commerce and SaaS clients, with algorithmic budget reallocation tools built in Python.
- Implemented Grafana + Prometheus observability for all production LLM deployments, providing clients real-time model health and cost dashboards.

Quantitative Analyst & Algorithmic Trader · Proprietary *2021 – Present | Remote*

- Develop and backtest quantitative trading strategies across XAU/USD, forex majors (EUR/USD, GBP/USD), BTC/USD, BankNifty, and Nifty50 using Python and TA-Lib.

- Built a multi-instrument signal engine combining momentum, mean-reversion, and volatility regime filters — achieving Sharpe ratio of 1.8+ in live trading.
- Automated trade execution and risk management with Python scripts integrating broker APIs, enforcing dynamic position sizing via Kelly Criterion and ATR-based stops.
- Apply time series models (ARIMA, GARCH, LSTM) for volatility forecasting and regime detection across commodity and equity index markets.
- Maintain live Grafana dashboards tracking P&L attribution, drawdown, win rate, and risk metrics across all instruments in real time.

AI/ML Engineer · **Freelance / Contract** *2022 – 2024 | Remote*

- Built enterprise-grade ML feature pipelines on Azure Databricks with PySpark, processing 100M+ records daily using Delta Lake ACID transactions.
- Delivered RAG systems with OpenAI embeddings + Pinecone/FAISS hybrid search, reducing LLM hallucination rates by 35% vs. vanilla GPT-4 baselines.
- Deployed LangChain agents with tool-use (web search, SQL, API calls) for autonomous workflow automation across five enterprise clients.
- Reduced model deployment cycles from 2 weeks to under 4 hours using Azure ML + MLflow experiment tracking and GitHub Actions CI/CD.

AI & ENGINEERING PROJECTS — CASE STUDIES

Enterprise RAG Knowledge Management Platform *[Python · LangChain · Azure OpenAI · Pinecone · FastAPI · React · Grafana]*

Scenario: A 2,000-person professional services firm had critical knowledge siloed across 40,000+ PDFs, Confluence pages, and SQL reports. Consultants spent 3–4 hours daily searching for institutional knowledge, resulting in $1.2M estimated annual productivity loss.

Approach:

- Designed a multi-source RAG ingestion pipeline normalizing PDFs, wikis, and structured SQL data into a unified embedding space using LangChain document loaders.
- Implemented hybrid dense-sparse retrieval (OpenAI embeddings + BM25) with cross-encoder re-ranking, achieving 92% retrieval precision@5.
- Built a streaming FastAPI backend with LangChain RetrievalQA chains, delivering sub-2-second responses with citation traceability to source documents.
- Instrumented the full pipeline with Grafana dashboards tracking query latency (P95), retrieval quality scores, token costs, and daily active users.
- Deployed on Azure AKS with auto-scaling, sustaining 500 concurrent users with 99.9% uptime.

Outcome: Reduced average knowledge lookup time from 3.5 hours to 8 minutes. Estimated annual productivity savings of $900K. Client expanded platform to 3 additional business units within 60 days of launch.

Automated ML Feature Engineering Pipeline on Azure Databricks *[Python · Databricks · Apache Spark · Delta Lake · Azure ML · MLflow · Azure Data Factory]*

Scenario: A fintech company's data science team spent 70% of sprint cycles on manual feature engineering for credit risk models, with no lineage tracking and feature inconsistencies causing model performance drift of 12% month-over-month.

Approach:

- Designed a reusable PySpark feature library on Databricks processing 100GB+ of daily transactional data with Delta Lake ACID guarantees and time-travel for reproducibility.
- Built Azure Data Factory orchestration pipelines with automated retraining triggers based on data drift thresholds, monitored via custom Grafana alerts.
- Integrated MLflow experiment tracking across 200+ model runs, enabling one-click model comparison and governed promotion to production.
- Implemented feature store pattern on Databricks, exposing 150+ pre-computed features as a shared service across 6 ML teams via Delta Live Tables.
- Set up data quality checks using Great Expectations, automatically failing pipelines on schema drift or null rate violations above 0.5%.

Outcome: Reduced feature development time by 80%, from 3 weeks to 3 days per feature set. Eliminated model performance drift — accuracy variance dropped from 12% to 1.4% month-over-month. Enabled 4x faster model iteration cycles.

LLM Operations & Cost Observability Stack *[Grafana · Prometheus · Python · OpenTelemetry · Docker · LangSmith · Azure Monitor]*

Scenario: An AI product company running 8 LLM-powered microservices had zero visibility into per-request token costs, retrieval quality, or model latency distribution. Monthly OpenAI bills exceeded budget by 40% with no attribution to individual features or users.

Approach:

- Built custom Python OpenTelemetry exporters instrumenting every LangChain chain, RAG retrieval step, and LLM call with structured spans and cost metadata.
- Designed a 12-panel Grafana dashboard covering: token spend by model/feature/user, P50/P95/P99 latency, retrieval precision trends, error rate, and cache hit ratio.
- Implemented semantic caching with Redis to deduplicate near-identical LLM queries — tracking cache savings live in Grafana.
- Configured multi-tier Prometheus alerting: cost anomaly detection (>20% budget spike triggers PagerDuty), latency SLO breach alerts, and retrieval quality degradation warnings.
- Integrated LangSmith traces for qualitative debugging alongside quantitative Grafana metrics, creating a unified observability layer.

Outcome: Identified 34% of token spend attributable to a single poorly-prompted feature. Cost reduced by $18K/month after prompt optimization guided by observability data. MTTD for model degradation fell from 4 hours to 9 minutes.

Conversational AI Voice Agent for Inbound Sales *[Python · LangChain · ElevenLabs · Twilio · Azure Functions · FastAPI · OpenAI]*

Scenario: A B2B SaaS company received 400+ inbound sales inquiry calls per week. SDRs handled only 60% within SLA, losing an estimated 30% of leads to competitor response speed. Hiring more SDRs was cost-prohibitive at $85K/head annually.

Approach:

- Architected a real-time voice agent pipeline: Twilio STT → intent classification (fine-tuned GPT-4) → LangChain agent with CRM tool-use → ElevenLabs TTS response.
- Designed a slot-filling conversation manager handling 18 distinct qualification intents, dynamically adjusting script based on company size, use case, and urgency signals.
- Implemented LangChain memory with sliding window + entity extraction to maintain coherent multi-turn conversations up to 15 minutes in length.
- Built a fallback escalation system routing complex objections to human SDRs with full conversation context, achieving seamless handoff in under 3 seconds.
- Deployed on Azure Functions with event-driven auto-scaling, handling concurrent call spikes with average LLM response latency of 180ms.

Outcome: Achieved 81% call containment rate (no human escalation). Lead response time reduced from 4 hours to under 45 seconds. Equivalent capacity of 6 additional SDRs at 12% of the annual hiring cost. Pipeline revenue attributable to agent grew 22% in first quarter.

QUANTITATIVE FINANCE PROJECTS — CASE STUDIES

Multi-Instrument Algorithmic Signal Engine *[Python · Pandas · NumPy · TA-Lib · Backtrader · Broker API · Grafana]*

Scenario: Manual discretionary trading across XAU/USD, BankNifty, and forex majors produced inconsistent results with high emotional bias. The challenge was building a systematic, backtested signal engine that could operate across uncorrelated asset classes with disciplined risk management.

Approach:

- Engineered a modular signal framework in Python combining momentum (ADX + EMA crossover), mean-reversion (Z-score of Bollinger Band deviation), and volatility regime filters (ATR-based classification: trending vs. ranging).
- Built a correlation matrix engine to detect regime shifts across asset pairs, dynamically reducing position size when cross-asset correlation exceeded 0.7 (indicating contagion risk).
- Implemented Kelly Criterion-based position sizing with ATR stops and 2% max drawdown per trade, with portfolio-level daily VaR constraint of 5%.
- Backtested 4 years of historical data (2020–2024) on XAU/USD, EUR/USD, BTC/USD, BankNifty using Backtrader with realistic slippage and commission modeling.
- Automated live signal generation and execution via broker API with Grafana P&L tracking, drawdown alerts, and real-time risk dashboard.

Outcome: Live strategy produced Sharpe ratio of 1.82, Sortino ratio of 2.41, and max drawdown of 11.3% vs. buy-and-hold benchmark of 27%. Win rate of 58% with 2.1R average reward-to-risk on winning trades.

Gold (XAU/USD) Volatility Forecasting Model *[Python · GARCH · LSTM · Scikit-learn · Pandas · Matplotlib · TA-Lib]*

Scenario: Gold price volatility clustering makes fixed stop-loss levels ineffective — tight stops get hunted during high-volatility sessions while wide stops over-risk capital in low-volatility periods. An adaptive, forecast-driven stop system was needed.

Approach:

- Assembled 10-year XAU/USD daily and hourly OHLCV dataset with macroeconomic features: DXY index, US10Y yield, CPI releases, COMEX futures positioning (COT data).
- Fitted GARCH(1,1) and EGARCH models for conditional volatility estimation, capturing asymmetric volatility response to positive vs. negative price shocks.
- Trained an LSTM sequence model on 60-bar rolling windows, predicting next-session ATR as a continuous regression target — outperforming GARCH by 18% on RMSE.
- Integrated forecast output into the signal engine as dynamic stop-loss multiplier: tight (1.2x ATR) in low-vol regime, wide (2.5x ATR) in high-vol regime.
- Built a Grafana panel visualizing rolling volatility forecast vs. realized volatility with regime state annotations for each session.

Outcome: Dynamic stop system reduced stop-out rate by 29% on winning trades and improved risk-adjusted returns by 0.4 Sharpe points vs. fixed ATR stops. Model forecasts are now core inputs to position sizing across all commodity trades.

BankNifty & Nifty50 Intraday Options Strategy with ML Regime Detection *[Python · Scikit-learn · Hidden Markov Models · TA-Lib · Options Pricing · NSE API]*

Scenario: Indian index options (BankNifty, Nifty50) exhibit strong intraday regime patterns — trending opens, consolidating midday, and volatile closes — yet most retail strategies apply uniform rules across all sessions, resulting in poor risk-adjusted returns.

Approach:

- Labeled 3 years of BankNifty 5-minute data into 3 market regimes (trend, range, breakout) using Hidden Markov Models trained on price action, volume, and IV features.
- Built regime-conditional strategy modules: trend regime triggers directional debit spreads; range regime sells iron condors targeting theta decay; breakout regime executes straddle momentum plays.
- Integrated real-time NSE option chain data via API to dynamically select strikes based on current IV rank, days-to-expiry, and delta targets.
- Built a backtesting harness accounting for options bid-ask spread, STT/transaction costs, and early exit triggers (50% profit / 100% loss rules).
- Monitored live Greeks exposure (delta, theta, vega) via Grafana with auto-alerts when portfolio delta exceeded ±200 or vega breached risk limits.

Outcome: Regime-conditional options strategy achieved 34% annualized return with max drawdown of 15.8% vs. naive straddle buy benchmark return of -8% over the same period. HMM regime classifier achieved 71% accuracy on held-out 2024 data.

AI-Powered Forex Sentiment & Macro Signal Dashboard *[Python · LangChain · OpenAI API · Grafana · Pandas · NewsAPI · FRED API]*

Scenario: Retail forex traders lack systematic access to real-time macro and sentiment signals that institutional desks use for EUR/USD, GBP/USD positioning. Manual news reading introduces latency and cognitive bias in fast-moving currency markets.

Approach:

- Built a real-time macro signal pipeline aggregating: FRED economic releases (CPI, NFP, PMI), central bank statement sentiment via OpenAI NLP classification, and live forex news via NewsAPI.
- Used LangChain to summarize and sentiment-score each news event (bullish/bearish/neutral per currency pair) with confidence scores, storing outputs in a time-series database.
- Engineered a composite macro score per currency pair combining: interest rate differential, economic surprise index, COT positioning, and NLP sentiment — updated every 15 minutes.
- Visualized the full macro dashboard in Grafana: sentiment heatmap by currency, upcoming event calendar with historical market impact, and live composite score trend lines.
- Integrated alerts for extreme sentiment divergence (sentiment vs. price action discordance > 2σ) as a contrarian signal trigger.

Outcome: Sentiment-informed signals improved discretionary EUR/USD and GBP/USD trade win rate from 51% to 64% over 6-month live forward test. Dashboard is used as a daily briefing tool, reducing pre-trade research time from 45 minutes to under 5 minutes.

EDUCATION

Bachelor of Engineering — Computer Science / Information Technology *2016 – 2020*

Hyderabad, India

CERTIFICATIONS

- Microsoft Certified: Azure Data Engineer Associate (DP-203)
- Databricks Certified Associate Developer for Apache Spark
- LangChain for LLM Application Development — DeepLearning.AI
- Building Systems with the ChatGPT API — DeepLearning.AI
- Financial Markets — Yale University / Coursera
- Algorithmic Trading & Quantitative Analysis — Udemy / QuantInsti

DEVOPS & INFRASTRUCTURE PROJECTS — CASE STUDIES

Containerised AI Microservices Platform with Docker & Kubernetes *[Docker · Docker Compose · Kubernetes (AKS) · Azure Container Registry · Nginx · FastAPI · GitHub Actions · Grafana]*

Scenario: ML Arc flagship AI product comprised 7 independent services — RAG API, LangChain agent orchestrator, vector ingestion worker, voice pipeline, authentication, Grafana observability stack, and a React frontend — all deployed manually on a single VM. Any update required full downtime, rollbacks were impossible, and environment drift caused build failures on every client handoff.

Approach:

- Containerised all 7 services into purpose-built Docker images using multi-stage builds, reducing final image sizes by 60% (Python API images: 1.8GB to 420MB) via distroless base layers and layer caching optimisation.
- Authored a production-grade docker-compose.yml for local development and staging: named volumes for vector store persistence, bridge networks isolating services, and health-check policies ensuring dependent services await upstream readiness.
- Parameterised all environment-specific config via .env files and Docker secrets — zero hardcoded credentials across all 7 images, passing OWASP secret scanning gates in CI pipeline.
- Migrated production to Azure Kubernetes Service (AKS): authored Helm charts per service with configurable replica counts, resource requests/limits, liveness/readiness probes, and Horizontal Pod Autoscaler rules.
- Built GitHub Actions CI/CD pipeline: on every merge, images are built, tagged with Git SHA, pushed to Azure Container Registry, and deployed to AKS via rolling update with automatic rollback if readiness probe fails within 120 seconds.
- Instrumented all containers with Prometheus exporters and Grafana dashboards: container CPU/memory per pod, restart counts, OOM events, deployment frequency, and DORA metrics (lead time, change failure rate).
- Configured Nginx ingress controller with TLS termination (Let's Encrypt), rate limiting (100 req/min per IP), and path-based routing to all backend services from a single domain.

Outcome: Deployment time dropped from 45-minute manual process to 6-minute automated pipeline. Zero-downtime rolling deployments eliminated all maintenance windows. Platform handles 10x traffic spikes via AKS autoscaling. MTTR reduced from 2 hours to 18 minutes with container-level Grafana alerting.

Dockerised Quantitative Backtesting & Research Environment *[Docker · Docker Compose · Python · Backtrader · Jupyter · PostgreSQL · TimescaleDB · Grafana]*

Scenario: Running quant strategy backtests across XAU/USD, BankNifty, Nifty50, and BTC/USD on different machines produced inconsistent results due to library version mismatches (TA-Lib, Pandas, NumPy). Sharing research environments required hours of manual setup and still produced dependency conflicts.

Approach:

- Built a reproducible quant research Docker image on Python 3.11-slim, pre-installing TA-Lib (compiled from source in build stage), Backtrader, Pandas, NumPy, Scikit-learn, PyTorch, and Jupyter Lab — all pinned to exact versions via requirements.txt.
- Configured a docker-compose stack with three services: jupyter (research environment), postgres with TimescaleDB extension (historical OHLCV store), and grafana (P&L visualisation) — wired via internal Docker bridge network with persistent named volumes.
- Automated OHLCV data ingestion into TimescaleDB using a Dockerised Python ETL container pulling from broker APIs and NSE feeds on a scheduled cron job.
- Mounted strategy code as bind-mount volume: researchers edit locally in VS Code, changes reflect instantly inside the running container without any rebuild cycle.
- Published image to private Azure Container Registry — any team member spins up an identical, fully-loaded research environment with a single docker-compose up in under 90 seconds.

Outcome: Researcher onboarding dropped from 4-hour manual setup to 90 seconds. Backtest reproducibility reached 100% across Mac, Windows, and Linux. Strategy iteration cycles accelerated 3x as environment debugging was fully eliminated.

ADDITIONAL TECHNICAL KEYWORDS

Machine Learning · Deep Learning · NLP · Generative AI · GPT-4o · Claude · Llama · Mistral · HuggingFace · Semantic Search · Embeddings · Knowledge Graph · Agentic AI · Tool Use · Function Calling · Retrieval-Augmented Generation · Streamlit · Gradio · TensorFlow · PyTorch · ARIMA · GARCH · LSTM · Monte Carlo Simulation · Options Pricing · Black-Scholes · Greeks · Backtesting · Quantitative Research · Statistical Modeling · Regression · Classification · Anomaly Detection · Feature Engineering · Data Lakehouse · ETL/ELT · REST API · WebSockets · OAuth · JWT · Docker · Docker Compose · Docker Swarm · Container Orchestration · Helm · Kubernetes · Azure Container Registry · Multi-stage Build · Distroless Images · CI/CD Containerisation · Agile · Scrum · Product Management · B2B SaaS · Client Management · Startup · Growth Marketing · Conversion Rate Optimization